Iowa State University Software Engineering: AI Curriculum Update Proposal

With the rapid emergence of agentic workflows and the "probabilistic computer" paradigm, the Software Engineering curriculum must evolve to prepare students for the modern professional environment. The following proposal maps the core concepts from the new AI-driven software engineering landscape onto Iowa State University's existing SE course sequence, and proposes a new advanced course for students pursuing a more in-depth systems architect role.

The updates emphasize a bifurcated pedagogical approach:

1. **Pragmatic/Applied Track (Track B)**: Understanding how to effectively *use* AI constraints, budgets, and tools in daily workflows.
2. **Advanced/Systems Architect Track (Track A)**: Understanding how to *build* the underlying probabilistic systems, parse token economics at scale, and architect new agentic hardware/software.

Proposed Updates to Existing Courses

1. SE 1850: Problem Solving in Software Engineering

Current Focus: Basic programming concepts and problem-solving strategies. *Proposed AI Integration (Pragmatic Focus):*

- **Introduction to AI Models Typology:** Distinguishing between **Frontier Models** (cloud-based, high-capability) and **Open-Weight Models** (local, privacy-preserving).
- **Basic Token Economics & Context Windows:** Teaching students that AI memory is finite and costs money. Introduce the concept that a token is analogous to a syllable or word, and every prompt carries a financial and latency cost.
- **Prompting as Problem Solving:** Emphasizing how to constrain prompts to solve problems effectively rather than relying solely on manual syntax generation.

2. COMS 3210 / CPRE 3810: Computer Architecture and Machine-Level Programming

Current Focus: CPU design, instruction set architectures, assembly, and classic Von Neumann architecture. *Proposed AI Integration (Advanced Focus):*

- **Specialized AI Hardware & Probabilistic Compute:** Contrast classical CPU architectures with dedicated AI accelerators (NPUs, TPUs, custom silicon). Discuss how these architectures are specifically optimized for the parallel matrix operations and high-throughput memory constraints required by modern LLMs.
- **The LLM OS Paradigm:** Frame the LLM's architecture in OS terms: Natural language as the Instruction Set Architecture (ISA), context windows as ephemeral high-speed RAM, and RAG/Vector DBs as disk storage.

3. SE 3090: Software Development Practices

Current Focus: Agile methodologies, version control, CI/CD, and practical software creation. *Proposed AI Integration (Pragmatic Focus):*

- **Test-Driven Agent Development (TDAD):** Shift from writing all code manually to writing strict, deterministic test suites and interface definitions *first*, and using coding agents (like Cursor or

GitHub Copilot) to generate the implementation.

- **The Salary Inversion Problem:** Teach the real-world economics of agentic loops. If an autonomous agent enters an infinite loop, cloud API costs can quickly eclipse an engineer's salary. Teach developers how to mandate `max_iterations`, budget tokens per sprint, and use circuit breakers.
- **CI/CD LLM Validation:** Integrating small open-weight models to automatically analyze pull requests and check for performance regressions before merges.

4. SE 3170: Introduction to Software Testing

Current Focus: Unit, integration, and system testing for deterministic code. *Proposed AI Integration (Advanced & Pragmatic Focus):*

- **Testing Non-Deterministic Systems:** How do you test a system that doesn't return the exact same output twice? Introduce the concept of **Self-Certification**, where a probabilistic execution is heavily gated by deterministic validators.
- **Constrained Decoding:** Teaching students how to mathematically force an LLM to output valid syntax (like strict JSON schemas) using finite state automatons so it can be safely ingested by deterministic unit tests.

5. SE 3390: Software Architecture and Design

Current Focus: Client-server models, microservices, design patterns, and system scaling. *Proposed AI Integration (Advanced Focus):*

- **Model Sizing & Parameter Optimization:** Understanding the architectural trade-offs of parameter counts (e.g., 8B vs 70B). Evaluating VRAM requirements, latency, and throughput when selecting a foundation model for a specific service tier.
- **Retrieval-Augmented Generation (RAG) Architecture:** Designing systems that ground probabilistic models using deterministic enterprise data. Introduce semantic search, vector databases (PgVector, Milvus), and cosine similarity metrics.
- **Agentic Architecture & ReAct:** Moving beyond static control flow (if/else branching) to stochastic orchestration (Reasoning and Acting).
- **Multi-Fidelity Routing:** Designing routing gateways that dynamically send high-precision tasks to classical deterministic algorithms, and unstructured intent tasks to stochastic LLM endpoints.

Proposed New Course

SE 4XX/5XX: Engineering Probabilistic and Agentic Systems

Target Audience: Seniors and Graduate students aiming for Frontier Lab-level systems architecture and advanced AI engineering roles. *Course Description:* This course dives deep into the emerging execution environment where probabilistic AI models are placed in the critical path of software systems. Students will build full-scale agentic architectures, balancing deterministic engineering requirements (latency, security, exactness) against the probabilistic nature of modern machine learning.

Key Modules:

1. **Advanced Token Economics & State Management:** Strategies for context window optimization, semantic chunking, and calculating cost-to-latency ratios. Mitigating "attention degradation" in large contexts.
2. **Building Autonomous Agentic Workflows:** Implementing ReAct frameworks with robust tool-calling parameters. Building secure, isolated environments for agents to execute system calls.

3. **Decoupled Asynchronous AI Infrastructure:** Using message brokers (Kafka, RabbitMQ) to decouple high-latency LLM inference from user-facing request/response cycles.
4. **Stateful Caching for AI:** Implementing vector caches (Redis) to store embeddings or exact-match prompt responses to drastically reduce compute redundancy and token expenditure.
5. **Zero-Trust AI:** Evaluating coding assistants and managing the risk of proprietary data leakage using local, quantized open-weight models.